引入 Redis 性能的提效在哪?

引入 Redis 的性能提升主要体现在 **“减少慢操作”** 和 **“减少重复操作”** 上：

1. **减少数据库压力**：将高频读取的数据（如热点商品信息、用户会话）缓存起来，**避免对MySQL等磁盘数据库的频繁查询**，MySQL的磁盘I/O是主要瓶颈。
2. **降低计算复杂度**：将复杂计算结果（如排行榜、秒杀库存）缓存，**避免每次请求都进行重复计算。**
3. **加速响应速度**：内存访问（Redis）的速度是纳秒级，而磁盘访问（MySQL）是毫秒级，相差10万倍。直接从内存返回数据，API响应极快。

Redis 为什么快?

Redis 快的原因是一个组合拳，I/O多路复用是其中最关键的一环：

1. **基于内存**：所有操作都在内存中完成，这是速度的基础。
2. **高效的数据结构**：Redis 不仅仅是简单的 Key-Value 存储，它的 Value 可以是多种数据结构，设计了 SDS、跳跃表、压缩列表等非常适合内存操作的数据结构。

3. **单线程模型（核心工作线程）**：
   - 避免了多线程的上下文切换和竞争带来的开销：不需要考虑锁的问题，不会因为加锁/释放锁而浪费时间
   - **为什么单线程还能这么快?**
     - 因为它的性能瓶颈主要在于**内存和网络I/O**，而不是CPU。单线程模型已经足以高效地利用内存。多线程带来的上下文切换开销在内存操作面前反而得不偿失。
   - **注意**：Redis 在 6.0 之后引入了**多线程I/O**，但这里多线程主要用于**处理网络数据的读写和协议解析**，而**执行命令的核心模块仍然是单线程的。**这进一步释放了网络I/O的瓶颈，但核心优势依然是单线程内存操作

4. **I/O 多路复用（核心网络模型）**：
   - I/O 多路复用是一种允许单个进程/线程**同时监视多个文件描述符（如 Socket 连接）**，并在其中任何一个或多个变得“就绪”（可读或可写）时获得通知的机制。
   - **原理**：Redis 使用 epoll (Linux)、kqueue (BSD/Mac) 等系统调用，在一个线程中监控多个Socket连接。当某个Socket有事件产生（如可读、可写）时，内核会通知Redis进程，Redis再顺序处理这些事件。
   - **好处**：**用单个线程解决了海量客户端的连接问题**，避免了为每个连接创建一个线程的巨大开销，将网络I/O的消耗降到了最低。

数据结构

小红书
小红书号：49607377662

Redis 五大基础数据类型

数据类型	底层数据结构	适用场景
String（字符串）	**int 和 SDS（简单动态字符串）**	计数器、缓存、分布式锁
Hash（哈希表）	**压缩列表和hashtable**	用户信息存储
List（列表）	**双向链表或压缩列表**	消息队列、排行榜
Set（集合）	**HashMap或 intset（整数集合）**	标签系统、去重
Zset（有序集合）	**跳表（SkipList）或压缩列表（ZipList）**	排行榜、延时队列

String

String类型底层使用**SDS**，SDS根据字符串长度不同，采用多种结构体以节省内存

```
struct sdshdr {
    int len;       // 已用长度
    int free;      // 剩余空间
    char buf[];    // 字节数组
};
```

优势：

- **O(1) 复杂度获取长度**（通过 `len` 字段，避免遍历）。
- **动态扩容**：减少内存重分配（空间预分配 + 惰性释放）。
- **二进制安全**：可存储任意数据（如图片、`\0` 字符）。

SET

- **底层数据结构**
  - **Intset：**整数集合，内部是一个有序数组，元素按升序排序，且元素不重复，在元素比较小的时候可以使用，比较节约内存空间。
  - **Hashtable,** 就是我们常常说的键值对存储，插入，删除，查找的平均时间为o(1)，非常适合存储大型的非整数set数据

小红书
小红书号：49607377662

ZSet有序集合📌

Set（有序集合）使用了两种编码结构，根据元素数量和大小自动切换：

- **内部实现**：
  - **跳表（SkipList）+ dict (字典)**： 支持高效的范围查询和排序。
    - **多层级链表**：跳表本质上是一个**多级索引的有序链表**，上层链表跳跃较远，下层链表精确查找。
    - **查询效率**：每次查找**可以跳跃多个节点**，减少遍历次数，查询复杂度为 **O(logN)。**
    - **插入/删除高效**
      - **插入** 只需要调整部分索引，不像**红黑树**那样需要**复杂的旋转**操作。
      - **删除** 也只需要调整索引，不会影响整个结构的平衡。

> **Redis 跳表添加元素过程**
>
> 找到插入位置并更新索引
>
> 1. **找到合适的插入位置**：从**最高层索引**开始，逐层向下查找合适的插入位置。
> 2. **随机决定元素的层级**：通过**随机算法（以 1/2 概率决定是否进入上一层）**，插入元素可能出现在多层索引中。
> 3. **更新链表指针**：在找到的插入位置处，**修改前后节点的指针**，完成插入。

- **压缩列表（ZipList）：** 当元素数量少（`<128`）且每个元素值小（`<64`字节）时使用。它在内存中是一块连续的内存，节省空间。
- **场景**：**排行榜**：存储用户的得分或排名，支持按分数排序。
- **扩展：利用ZSet 实现延迟队列：**
  - **Sorted Set** 的每个成员都有一个分数，分数是浮动的，并且按分数排序。我们可以利用这一特性来为每个任务设置一个未来的时间戳作为分数。
  - **时间戳**：任务的延迟时间戳可以是 **当前时间 + 延迟时间。**
  - **处理逻辑**：任务的时间戳到达时，可以从队列中取出并执行任务

Hash（哈希表）

底层实现

ziplist

- 当 Hash 的 **字段数量少** 且 **字段值较小** 时，Redis 默认使用 ziplist 存储，
- **适用场景**：适合小数据量场景的线性遍历。。
- **数据结构**：连续内存块，按顺序存储字段名和字段值（无指针开销）。
  - **缺点**：
    - **修改效率低**：插入/删除需重新分配内存，时间复杂度 O(n)。
    - **查询效率低**：需遍历查找字段（时间复杂度 O(n)）。

小红书
小红书号：49607377662

hashtable

- 当 Hash 的字段数量或字段值长度超过阈值时，转为 hashtable，**查询高效**平均时间复杂度 O(1)。。
- **数据结构：**
  - 类似 Java 的 `HashMap`，数组 + 链表结构（Redis 7.0 后改为 **链式哈希**）。
  - 每个哈希节点存储 `field-value` 键值对。
  - **冲突解决**：链表法（同一哈希槽的节点用链表连接）。
  - **动态扩容**：支持渐进式 rehash：

#####

哈希扩容：渐进式Rehash

Redis 的 Hash 类型在扩容时使用 **渐进式 Rehash** 机制。当哈希表的负载因子超过阈值（默认是1）时，Redis 会创建一个大小为旧表两倍的新哈希表。**迁移过程中，新旧表同时存在**，每次操作（如读写请求）会迁移一个哈希槽的数据到新表，同时后台任务也会辅助迁移。这样做的好处是避免一次性迁移导致服务阻塞，确保高可用性。

转移期间，**读取操作会同时查询新旧表，而写入直接进入新表。**如果新表在迁移过程中再次满了，Redis 会继续扩容到更大的容量，直到负载达标。整个过程平滑高效，是 Redis 实现高性能和高可用的重要机制

扩容过程中如果有大量写怎么办？
答案就是：**得益于渐进式Rehash的设计，大量写请求会被正常处理，并且每个请求还会协助加速数据的迁移过程，不会导致服务不可用。唯一的副作用是此期间内存消耗会翻倍，需要提前规划好内存容量。**

场景题：设计关注功能

1. 存储关系（核心设计）

使用**ZSet**缓存用户的关注列表和粉丝列表，支持按时间排序

- key: follow_zset:{user_id} -> zset(score=timestamp, member=follow_user_id)
- key: follower_zset:{user_id} -> zset(score=timestamp, member=follower_user_id)

优点：可按关注时间排序，支持分页（ZRANGE） 推荐用于需要“最近关注”排序的场景

1. **为什么这是最佳选择？**
   - **天然分页**: 使用 `ZREVRANGE follows:zset:123 0 9 WITHSCORES` 可以轻松获取用户123最近关注的10个人及其关注时间。
   - **时间排序**: 无论是查看“我的关注”还是“我的粉丝”，按时间倒序排列是最常见的需求。
   - **判断关系存在**: `ZSCORE follows:zset:{A} {B}` 不仅可以判断A是否关注了B，还能返回关注时间。如果返回不为空，即已关注。

小红书
小红书号：49607377662

Redis 高级数据类型

HyperLogLog

> **底层实现**：基于 **概率统计** 估算集合基数，误差约 **0.81%**。
> **适用场景**：超大规模数据去重（无需存储具体数据）。

- **占用空间固定**：无论存入 **10 万** 还是 **10 亿** 个数据，大小约 **12KB**。
- **适合统计独立访问（UV）、去重**，但不适用于精准去重。

Geo（地理位置）

> **底层实现**：基于 **Zset**（跳表 + 哈希表）
> **适用场景**：存储和计算**地理位置数据**。

- 通过 **Geohash** 算法，将**经纬度**编码成一个 52-bit 整数。
- 提供**附近查找**、**两点距离计算**等操作。

Stream（流数据

> **底层实现**：高性能**消息队列**，类似 Kafka / RabbitMQ。
> **适用场景**：消息队列、日志存储、实时数据流处理。

- **持久化存储**（不像 Pub/Sub 只能**实时消费**）。
- **支持消费组**（多个消费者消费不同的消息）。
- **支持历史数据回溯**（不像 List 会丢失旧数据）。

BitMap（位图）📌

底层

用**最小存储空间**进行**二值状态存储**（0/1）
底层实现用 String 类型实现的一种统计**01二值状态**的数据类型。

应用场景

- **用户签到记录**：存储每个用户的签到状态，用 1 表示已签到，0 表示未签到。
- **统计分析**：记录大规模的数据分析，比如用户行为、IP访问记录等。
- **去重**：通过 BitMap 来记录某些大数据量的唯一标识符，比如日志中的 IP 地址、用户 ID 等。
- 🟢**布隆过滤器**：BitMap 常常用作布隆过滤器的数据存储结构之一，在处理大规模数据去重时，能快速判断某个元素是否在集合中。

布隆过滤器📌

- 用于**快速判断元素是否存在**，**允许误判但不会漏判**。
- 适用于**黑名单系统**、**缓存预热**、**推荐系统**等。

底层原理：

小红书
小红书号：49607377662

1. 采用**位数组（BitMap）**存储数据，每个元素用 **多个哈希函数** 映射到不同的位置，将这些位设为 `1`。
2. 查询时，对目标元素进行相同的哈希计算，**如果所有对应的位置都是 `1`，则“可能存在”**；若有**任何一个位为 `0`，则“一定不存在”**。

数据越来越多如何维护？

- 布隆过滤器的**位数组大小是固定的**，当元素数量增多时：
- **缺点**：
  - **无法删除元素**：因为位上的1可能是多个元素共同映射的结果，删除一个元素（将1置0）可能会影响其他元素的判断。解决方案是使用**计数布隆过滤器**（用计数器代替位），但会占用更多空间。
  - **空间利用率**：虽然比HashMap省空间，但要达到较低的误判率，仍然需要分配一个较大的位数组。
  - **查询性能**：查询性能是O(k)（k是哈希函数个数），但k的大小会影响误判率，需要权衡。
- **解决方案**：
  - **扩容**：创建新的布隆过滤器，将旧数据搬迁。
  - **分层**：多个布隆过滤器并存，随着数据增长创建新的布隆过滤器，老数据慢慢过期。
  - **计数布隆过滤器**：用 `4-bit` 计数代替 `1-bit`，支持删除操作。

布隆过滤器误判率如何权衡？

布隆过滤器是**概率型数据结构**，允许**一定误判**（即可能误判“存在”，但不会误判“不存在”）

布隆过滤器的误判率可以通过公式计算，即 P = (1 - e^(-kn/m))^k，其中 k 是哈希函数的数量，n 是插入元素的数量，m 是布隆过滤器位数组的大小

误判率由 **哈希函数数量 `k` 和位数组大小 `m` 影响**：

增大位数组 `m` → 误判率降低（但占用更多内存）

增加哈希函数 `k` → 误判率先降后升（过多会增加查询时间）

优化哈希函数 → 选用合适的哈希算法，减少哈希碰撞

Redis 单线程指的是「接收客户端请求->解析请求 ->进行数据读写等操作->发送数据给客户端」

这个过程是由一个线程（主线程）来完成的

但是，**Redis 程序并不是单线程的**，Redis 在启动的时候，是会**启动后台线程**（BIO）的：

- **Redis 在 2.6 版本**，会启动 2 个后台线程，分别处理关闭文件、AOF 刷盘这两个任务；
- **Redis 在 4.0 版本之后**，新增了一个新的后台线程，用来异步释放 Redis 内存

小红书
小红书号：49607377662

热点数据 & 大数据量的处理

- **其他数据结构**：
  - **布隆过滤器 (Bloom Filter)**：快速判断某个数据**一定不存在**或**可能存在**于海量数据中，常用于缓存穿透场景。
  - **HyperLogLog**：用于做**基数统计**（估算一个集合中不重复元素的个数），占用空间极小（12KB），误差率约0.81%。
  - **位图 (Bitmap)**：用位数组来存储状态（如用户签到记录），极其节省空间。
- **数据较大怎么办**：
  1. **分片 (Sharding)**：将大数据集分布到多个 Redis 实例上（如Redis Cluster）。
  2. **淘汰策略 (Eviction Policies)**：配置 `maxmemory-policy`（如 `allkeys-lru`），当内存不足时自动淘汰数据。
  3. **过期时间**：给Key设置TTL，让其自动过期。
  4. **数据冷热分离**：将热数据放在Redis中，冷数据持久化到数据库或其他存储中。

缓存

过期策略

定期删除+惰性删除

日常的过期键清理，redis采用的是定期删除+惰性

- **触发时机**：在访问一个键时（如 `GET`，`SET` 等命令），Redis 会先检查该键是否设置了过期时间，以及是否已过期。
- **优点**：对 CPU 最友好。只有在必须的时候才进行检查和删除，不会在无关的键上浪费 CPU 时间。
- **缺点**：对内存不友好。如果一个键永远不再被访问，那么即使它已经过期，也会一直占用着内存，造成**内存泄漏**。这是导致需要“内存淘汰机制”的直接原因。

1. **定期删除**
   - **触发时机**：由 Redis 的后台进程**周期性**地执行。
   - **工作模式**：
     - **不是全量扫描**：正如您指出的，Redis 不会扫描所有键，那样会卡死。
     - **是抽样扫描**：它会**随机**地从设置了过期时间的键空间中抽取一定数量的键（默认是20个），检查并删除其中的过期键。
     - **是自适应循环**：如果一轮抽样中删除的过期键比例很高（超过25%），说明过期键还很多，它会立即**继续下一轮抽样**，直到过期键的比例降下来。这确保了能快速清理大量的集中过期键。
   - **优点**：通过周期性的工作，在一定程度上缓解了“惰性删除”造成的内存泄漏问题。
   - **缺点**：它仍然是一个**抽样**过程，无法保证所有过期键都会被及时删除。

但是，这两种策略都不是完全精准和及时的。 可能会有一些过期键既没有被定期删除抽中，也一直没被访问过，从而长期驻留在内存中。

小红书
小红书号：49607377662

所以，第二步，就是当导致内存使用达到上限时，触发的最终手段——内存淘汰机制。 这时，Redis 会根据我们配置的 `maxmemory-policy` 来决定怎么做，比如：

- **最常用的是 `allkeys-lru`，它在所有键中淘汰最近最少使用的。**
- 如果所有数据都可以被淘汰，就用 `allkeys-lru` 或 `allkeys-lfu`。
- 如果有些核心数据不能动，只淘汰缓存数据，可以用 `volatile-lru`。
- 最保守的是 `noeviction`，它直接拒绝写入，确保数据不丢失。

内存淘汰策略

当内存达到 `maxmemory` 时，Redis 提供 8 种可配置的淘汰策略：

1. 不淘汰，直接报错

- `noeviction`：拒绝所有会申请额外内存的写命令（如 `SET`，`LPUSH` 等），并返回错误。读命令正常。**最安全，适用于不允许丢数据的场景。**

2. 在所有键中淘汰

- `allkeys-lru`：从**所有键**中，淘汰 **最近最久未使用** 的键。**最通用、最推荐**的策略。
- `allkeys-lfu` (v4.0+)：从**所有键**中，淘汰 **访问频率最低** 的键。适用于区分热点和冷数据。
- `allkeys-random`：从**所有键**中，**随机** 淘汰一个键。性能好，但精度低。

3. 在设置了过期时间的键中淘汰

- `volatile-lru`：从**设置了过期时间的键**中，淘汰 **最近最久未使用** 的键。
- `volatile-lfu` (v4.0+)：从**设置了过期时间的键**中，淘汰 **访问频率最低** 的键。
- `volatile-random`：从**设置了过期时间的键**中，**随机** 淘汰一个键。
- `volatile-ttl`：从**设置了过期时间的键**中，淘汰 **剩余存活时间最短** 的键。最直观，优先淘汰快过期的数据

如何优化?

1. **合理分配 `maxmemory`**，防止 OOM：
2. **设置合理的过期时间（TTL）**，让数据自动清理：
3. **避免大 Key（Big Key）**，影响淘汰效率：

小红书
小红书号：49607377662

Redis 和数据库一致性 📌

更新策略：Cache Aside（旁路缓存）策略

- **主动更新**：数据库数据变化时，主动更新缓存。
  - **Redis 和 MySQL 的更新策略用的是 Cache Aside**，另外两种策略应用不了。
    - **读时：查询缓存，未命中再查数据库，并写入缓存**
    - **写时：先更新数据库，再删除缓存**
      - 避免直接更新缓存，会出现缓存和数据库的数据不一致性的问题
- **延迟双删**：先删除缓存、更新数据库、500ms 后再次删除缓存，防止并发请求导致数据不一致
- **容错设计**：
  - 可能存在删除缓存失败的问题，提供一个补偿措施即可
  - **超时剔除兜底**：
    所有缓存均设置超时时间（如30分钟），即使主动删除失败，旧数据也会自动过期。
  - **操作失败重试**：
    若删除缓存失败，记录日志并通过消息队列异步重试（如RocketMQ事务消息）。

缓存穿透雪崩击穿 📌

缓存穿透

- **定义**：缓存穿透指的是请求的数据**既不在缓存中也不在数据库中**，发现数据库中也没有要访问的数据，没办法构建缓存数据，来服务后续的请求

  产生原因：请求的数据在缓存和数据库中都不存在，或者**查询条件本身是错误的**（例如，非法用户 ID 等）/缓存没有进行有效的空值缓存，导致查询不到的数据仍然被请求到数据库。
- **解决方案**：
  - **使用布隆过滤器快速判断数据是否存在，避免通过查询数据库来判断数据是否存在**：请求只会查询 Redis 和布隆过滤器，而不会查询数据库，Redis 自身也是支持布隆过滤器的
  - **设置空值或者默认值**：对于查询结果为空的数据（例如，查询不到的用户信息），可以缓存一个特定的标记（如 `null` 或者某个特殊值），防止相同的无效请求再次访问数据库。

缓存雪崩

- **定义**：缓存雪崩指的是在短时间内，**大量缓存**失效或不可用，导致大量请求直接访问数据库
- **解决方案**：
  - **缓存过期时间设置随机值**：避免大量缓存同一时刻过期给缓存设置过期时间
    - 可以给缓存的过期时间加上随机偏移量

小红书
小红书号：49607377662

- **使用双重缓存策略**：即同时使用本地缓存和 Redis 缓存，保证 Redis 出现问题时可以由本地缓存提供数据，减少对数据库的访问。

缓存击穿

- **定义**：**单个热点 key 失效**，大量请求同时访问该 key 并穿透到数据库。
- **解决方案**：
  - **使用互斥锁**：
    - 当发现缓存中没有数据时，先尝试获取分布式锁（如 `SETNX` 或者 `Redisson`）。

      只有获取到锁的请求去数据库查询并写回缓存。

      其他请求等待或自旋，等缓存更新后再读取。
  - **预加载/提前刷新缓存**：对于一些热点数据，可以采用预加载机制，在数据即将过期时提前加载新的数据

持久化✅

Redis是内存数据库，数据存储在内存中进行读写操作

持久化机制确保数据在服务器重启或故障后不丢失，将内存数据保存到磁盘中，这样在 Redis **重启就能够从磁盘中恢复原有的数据。**

RDB

原理：通过生成内存数据的**二进制快照文件**，保存某个时间点的全量数据。

- 在指定时间间隔内，fork一个子进程，将内存中的数据**快照**写入一个临时RDB文件，替换旧的。

触发方式：

- **手动触发**：
  - `SAVE` 命令：**同步阻塞**主进程生成快照，期间无法处理任何请求。

小红书
小红书号：49607377662

- `BGSAVE` 命令：**异步非阻塞**，通过 `fork` 子进程生成快照，主进程继续处理请求。
- **自动触发**：
  - 配置 `save <seconds> <changes>` 规则
  - 主从复制时，主节点生成 RDB 发送给从节点。

工作流程

1. **fork 子进程**：主进程调用 `fork()` 创建子进程，**共享内存数据。**
2. **写时复制（Copy-on-Write, COW）**：

> 在fork过程中，操作系统会采用**写时复制机制**，使得父子进程在初始状态下共享内存页，只有在写操作发生时才实际分配新的内存页。这种机制可以显著减少内存开销并提高效率。

   - 子进程遍历内存数据，序列化写入 RDB 文件。
   - **主进程对新写入的数据**：在内存中创建副本，新数据仅影响主进程内存，原数据仍由子进程写入 RDB。
3. **替换旧文件**：子进程完成写入后，用新 RDB 文件替换旧文件

优缺点

- **优点**：
  - **文件紧凑**：二进制格式，体积小，适合备份和灾难恢复。
  - **恢复速度快**：直接加载到内存，比 AOF 快。
  - **性能影响小**：异步 `BGSAVE` 对主进程影响极小。
- **缺点**：
  - **数据丢失风险**：两次快照之间的数据可能丢失（取决于触发频率）。
  - **大数据量时 fork 耗时**：数据量较大时，`fork` 可能阻塞主进程（如内存 10GB，fork 耗时约 100ms）。

AOF

核心机制

- **原理**：**AOF 通过记录每一次写操作命令来持久化数据**（如 `SET`，`DEL`）到日志文件，当 Redis 重启时，它会**重新执行一遍 AOF 文件中的所有命令**，在内存中重建整个数据集，恢复到宕机前的状态。
- **开启方式**：在配置文件redis.conf中设置 appendonly yes，设置不同的同步策略，
  - `always`（每次写入同步）、`everysec`（每秒同步，**默认**生产环境常用）、`no`（操作系统决定）

AOF 重写 (AOF Rewrite)

随着服务器运行，AOF 文件会越来越大，因为它记录了所有的写操作。

小红书
小红书号：49607377662

AOF文件过大处理：通过执行 `BGREWRITEAOF` 命令（或自动触发）进行**AOF重写。**

- **原理**：Redis会fork一个子进程，根据当前数据库的数据状态，重新生成一个新的、更小的AOF文件。这个新文件包含了恢复当前数据状态所需的**最小命令集合**（例如，对一个key的多次写操作会被合并为最终状态的一条命令）。生成完成后，会用新文件替换旧文件。

AOF重写利用写时复制技术，确保在重写过程中父进程可以继续处理客户端请求，而子进程专注于生成新的AOF文件。即使父进程在重写期间进行了写操作，由于写时复制的存在，**父进程的修改不会影响子进程正在使用的内存数据。**

1. **Fork子进程**：当Redis执行AOF重写时，它会fork一个子进程来处理重写任务。父进程和子进程在fork之后共享相同的内存页。只有当某个内存页的内容需要被修改时，才会触发**写时复制**，为该页创建一个独立的副本。

   > “`fork` 创建子进程时，子进程与主进程**共享物理内存**。通过**写时复制（Copy-On-Write）**机制，只有在主进程修改数据时，操作系统才会复制被修改的**内存页**。这保证了子进程能看到 `fork` 时刻的**数据一致性的快照**
   >
   > - **子进程**（执行AOF重写或RDB快照）只需读取数据。由于数据未被修改，它始终访问共享的原始内存页，看到的是 `fork` 瞬间的**数据一致性快照**。
   > - **主进程**可以继续处理写请求。每次写操作修改数据时，仅复制相关的内存页，不影响子进程的快照视图。

2. **子进程写新文件**：子进程遍历内存中的所有数据，将每个键值对的**当前状态**用最精简的命令写入一个临时新文件。
3. **主进程缓冲新命令**：重写期间，主进程将新的写命令同时记录到**原有的AOF缓冲区**和**重写缓冲区**中。
4. **收尾与替换**：子进程完成后，主进程将**重写缓冲区**的命令追加到新文件，然后**原子性地替换**旧AOF文件。

AOF 的优缺点

- **优点**：**数据丢失风险更低**
- **缺点**：
  - **文件体积大**，比 RDB 方式占用更多磁盘空间。
  - **恢复速度比 RDB 慢**，因为需要逐条重放日志。
  - 需要**定期重写（Rewrite）**，否则文件会无限增长。
- **适用场景**：**高可靠性要求**的场景，如**金融交易、订单处理等数据不可丢失的业务。**

混合持久化

- **原理**：Redis 4.0+ 版本支持**混合持久化**，即**RDB 快照 + AOF 追加日志。**
- **优点**：**同时具备 RDB 启动快 & AOF 数据丢失少的优点。AOF 体积更小**，减少磁盘 I/O 开销。
- **缺点**：**比单独的 RDB 稍占空间，比单独的 AOF 稍慢。**
- **适用场景**：**希望在启动时加快恢复速度，又希望减少数据丢失**的业务。

小红书
小红书号：49607377662

如何发现 RDB 持久化故障

1. **检查 Redis 日志**
   - Redis 的日志（`redis.log`）可能会包含 RDB 失败的原因，如磁盘空间不足、权限问题等。
2. 查看持久化信息
   - 运行 `INFO Persistence` 获取 RDB 相关信息：redis-cli INFO Persistence

解决方案

1. **磁盘空间不足**：检查磁盘使用情况：df -h，清理无用文件，释放空间。
2. **权限问题**：确保 Redis 进程有写入 RDB 文件的权限：
3. **内存问题**Redis 可能因为内存不足无法 fork 进行 RDB：free -m
   - 调整 `overcommit_memory`
   - 增加 Swap：

Redis部署 ✅

常见部署架构

部署方式：

- **单机模式**：最简单，用于开发测试。存在单点故障风险。
- **主从复制 (Master-Slave)**：一主多从，主负责写，从负责读和备份。**读写分离**提升读性能，**数据冗余**保证高可用。
- **哨兵模式 (Sentinel)**：在主从基础上，增加了哨兵进程来监控主节点。主节点宕机时，哨兵能**自动完成故障转移**（选举新主），实现**高可用**。
- **集群模式 (Cluster)**：官方提供的**分布式方案**。通过**分片（Sharding）** 将数据分散到多个主节点上存储，每个主节点对应一个或多个从节点。兼具**高可用**和**大数据量扩展**能力。
  - **分片如何做**：
    - Redis Cluster 采用 **哈希槽（Hash Slot）** 进行分片。
    - 整个集群有 **16384** 个槽位。每个键通过 `CRC16(key) % 16384` 计算出一个槽位编号。
    - 集群将槽位分配给各个主节点负责。增加或删除节点时，只需移动部分槽位和数据即可，**无需重启集群**。

小红书
小红书号：49607377662

主从复制模式✅

在 Redis 的主从模式中，**一个主节点（Master）** 负责写入，多个 **从节点（Slave）** 负责读取，主节点将数据同步到从节点

- **高可用性**：当主节点发生故障时，可以通过从节点快速进行故障转移，确保服务持续可用。
- **读写分离**：主节点处理写请求，从节点处理读请求，减轻主节点压力，提升读取性能。
- **优点**：读写分离，提高性能，适合高并发读的业务。
- **缺点**
  - **Master 挂掉时需要手动切换**，恢复慢。
  - **一致性问题**：数据同步有延迟，可能读到旧数据。

主从数据同步模式

全量同步

- **首次连接**时，主库**生成 RDB 快照**，发送给从库。
- 在生成快照期间，主节点会将新写入的命令缓存在「复制缓冲区」；
- 从节点加载 RDB 后，再接收并执行缓冲区里的命令，保持数据一致。

部分复制

- 主库会**记录最近的命令（Replication Offset）**，从库保存了主节点的复制ID和偏移量
- 只同步最近的**数据变更**，避免全量同步带来的性能开销。
- **过程**：
  1. 连接恢复后，从节点告诉主节点：我上次同步到 offset X；
  2. 主节点检查复制缓冲区是否还有从 X 开始的命令；
     - ✅ 有：直接增量同步（无需发送全量 RDB）；
     - ❌ 没有：回退到全量复制。

  📌 Redis 使用 **复制积压缓冲区（Replication Backlog）** 来保存最近的写操作数据，默认大小 1MB（可配置 `repl-backlog-size`）。

主从同步的问题

- 数据丢失（主库宕机）
  - Redis 采用**异步复制**，主库可能还没来得及把最新数据同步到从库就发生宕机。
    - 开启 `wait` 命令，等待从库确认写入后再返回成功。
    - 使用 **Redis Sentinel 或 Redis Cluster**，实现高可用。
- **全量复制开销大**
  - **主库重启**，所有从库同时发起全量同步（Full Sync），**造成主库负载过高。**
    - **多级复制（Master → Slave1 → Slave2）**，减少主库压力。
    - **持久化 RDB**，避免主库频繁重新生成快照。
- **主从数据不一致**

小红书
小红书号：49607377662

- 由于 Redis 主从同步是**异步的**，某些写入可能还未同步到从库，主库就宕机，导致数据丢失。
  - 使用 `wait` **命令**，确保写操作**至少复制到 N 个从库**。
  - 启用 **AOF（Append Only File）**，确保数据不丢失

主从对分布式锁的影响

当 Redis 采用 **主从架构** 时，数据从主库同步到从库，但同步存在**延迟**，这会影响分布式锁的正确性。

> **异常场景**
>
> - 进程 A 在主库加锁，但**从库尚未同步到最新数据**。
> - 进程 B 读取**从库**，发现锁不存在，以为自己可以加锁。
> - 进程 B 在主库**再次加锁**，导致**两个进程同时持有锁**。

解决方案：使用 RedLock 或基于 ZooKeeper 的分布式锁，确保锁信息可靠

- **RedLock 机制**
  - 需要**多个 Redis 实例（通常 5 个）**，分别尝试获取锁，如果大多数（如 3/5）成功，则认为加锁成功。
- **RedLock 问题**：
  - **网络分区问题**：如果部分节点**因网络问题无法访问**，RedLock 可能仍然会认为加锁成功，造成数据不一致。
  - **时钟漂移问题**：不同 Redis 实例的**时间不同步**，可能导致锁提前或延后过期，影响锁的安全性。
  - **比单机锁复杂**：在大部分应用场景下，**单 Redis + SETNX 方案已经足够。**

哨兵模式

哨兵模式——✅监听探测

- **原理**：哨兵本身是**一个独立的 Redis 进程**，用于监控 Redis 主从架构并实现**自动主从切换。**。
- **作用：**
  - **监控（Monitor）**：实时检测 Master 和 Slave 状态
  - **自动故障转移（Failover）**：当 master 异常时，自动将某个 slave 提升为 master，并通知其他 slave
  - **通知（Notification）**：通知客户端新的 Master 地址
  - 缺点:仍然存在 **写操作只能在主节点** 的问题。**Sentinel 需要运行在多个实例上**，本身也是分布式系统

小红书
小红书号:49607377662

哨兵工作流程

1. **健康监测**：哨兵每秒 PING 定期 ping 主节点，判断是否宕机。
2. **故障判定**：超过一定时间未响应，则认为 Master 宕机。
3. **投票选举**：多个 哨兵 通过**Raft 选举算法**选择新 Master
   - Raft 通过 **多数派原则** 来避免脑裂现象：
   - **防止多个 Master 出现**（脑裂），如果投票未通过，重新发起投票
4. **自动切换**：所有的从节点会自动切换到新的 Master，并同步数据

Raft 选举算法

Raft 主要由以下**三个角色**组成：

- **Leader（领导者）**：负责处理客户端请求，并同步日志到 Follower。
- **Follower（跟随者）**：被动接收 Leader 的指令，不主动发起请求。
- **Candidate（候选者）**：在 Leader 失效时发起选举，尝试成为新的 Leader。

Raft **保证在任何时刻只有一个 Leader**，其他节点作为 Follower 进行数据复制。

2. Raft 的核心流程

(1) Leader 选举

- 当 Follower 在**一定时间内没有收到 Leader 的心跳**，就会变成 Candidate，并发起选举：
  1. Candidate 递增 **当前任期（term）** 并向其他节点请求投票。
  2. 若 Candidate 获得多数票（majority），则成为 Leader。
  3. 新 Leader 立即向所有 Follower 发送心跳，防止再次选举。

> **关键机制：**
>
> - **随机超时**（避免选举冲突）：每个 Follower 触发选举的时间是**随机的**，防止多个节点同时成为 Candidate 造成分裂。
> - **投票机制**：Follower 只能投**一次**票，且只能投给**比自己日志更新的 Candidate**。

==**Redis 社区版的新探测模式——Gossip 协议**==

Paxos 算法

“Paxos 是一种分布式一致性算法，用来保证在部分节点故障的情况下，多数派节点依然能对同一个值达成一致。

- **机制**：核心是1. Paxos 算法

小红书
小红书号：49607377662

面试回答示例：
“Paxos 是一种分布式一致性算法，用来保证在部分节点故障的情况下，多数派节点依然能对同一个值达成一致。

- **机制**：核心是两阶段投票：
  1. **Prepare 阶段**：提议者（Proposer）发起提案，请求多数派接受者（Acceptor）承诺不接受编号更小的提案。
  2. **Accept 阶段**：如果得到多数派承诺，提议者再发出正式提案，多数派接受后就达成一致。

Redis Cluster集群

Redis Cluster 是 Redis 官方提供的**分布式数据库解决方案。**

将数据分布在多个节点上，每个节点负责一部分数据的存储和读写。节点之间通过**Gossip协议**进行通信，自动发现和维护节点信息。Redis Cluster 支持动态添加和删除节点，具备高可用性和可扩展

它通过**分片（Sharding）** 来进行数据分布，并提供**复制（Replication）** 和**故障转移（Failover）** 的能力，实现了在高可用性基础上的横向扩展。

在redis集群中，主节点接收各种来自客户端的需求，

- **读操作**，主节点会根据哈希槽找到对应的子节点进行处理，并将处理结果返回客户端，
- **写操作**，主节点会进行本地处理，保证数据的一致性和正确性，随后将写操作发送给所有子节点进行复制，主从复制确保主节点和子节点的数据一致

数据分片：哈希槽

这是 Redis Cluster 最核心的设计。

哈希槽（Hash Slot）的分区方法

- **槽的数量**：整个集群被划分为 **16384** 个槽。这个数字是精心选择的，足够大以便于均匀分布，又足够小以便于节点间交换配置信息（心跳包中可以携带全量的槽配置）。

  > 为什么选择 16384 个槽？**每个节点需维护所有槽的分配信息。若槽数为 16384，每个节点仅需约** 2KB 内存 存储槽位图（`16384 bits = 2048 bytes`）

- **数据映射**：每个键都属于这 16384 个槽中的一个
  - 使用 **CRC16** 算法计算键的哈希值,再对哈希值取模：`slot = CRC16(key) % 16384`。
- **槽分配**：将这 16384 个槽**分配**给集群中的主节点。
- **关键**：**只有当所有 16384 个槽都被分配后，集群才会进入 `OK` 状态**，才能正常提供服务。否则集群会处于 `FAIL` 状态。

哈希槽将数据集划分为固定数目的槽位（ **16384 个哈希槽**），主节点通过对键进行哈希运算得到对应的槽位编

小红书
小红书号：49607377662

集群如何实现高可用

A： 通过 **主从复制 + 故障自动转移** 机制。

- **主从结构**：每个负责槽的主节点，都应该有一个或多个从节点。
- **数据同步**：从节点异步复制主节点的数据。
- **故障检测**：集群中的每个节点都会与其他节点通过 **Gossip 协议** 进行通信，互相发送 PING/PONG 消息来检测节点是否下线。
  - **主观下线**：一个节点认为另一个节点不可用。
  - **客观下线**：当超过半数的**主节点**都认为某个主节点主观下线时，整个集群就将其标记为客观下线，并触发故障转移。
- **故障转移**：当主节点被标记为客观下线后，其下的一个从节点会升级为新的主节点（通过选举机制）。这个过程是自动的，客户端几乎无感知。

节点通信：Gossip协议

节点之间通过**Gossip协议**进行通信

Gossip 协议是一种去中心化的、最终一致性的通信协议。它的工作方式就像“流言传播”或“流行病扩散”。

工作原理：

1. **周期性地传播**：每个节点周期性地（例如每秒）从自己已知的节点列表中**随机选择**一个节点。
2. **交换信息**：节点之间通过 PING/PONG 消息进行通信，这些消息中会携带**元数据**（如集群节点列表、槽映射、节点状态等）。
3. **感染式传播**：节点 A 将自身的信息告诉节点 B，节点 B 就更新了自己的信息；然后节点 B 再和随机的节点 C 通信，将信息（包括从 A 那里听来的）传播出去。
4. **最终一致性**：经过一段时间后，集群中的所有节点都会获得一份趋于一致的集群状态视图。

优点：可扩展性好、容错性高（任何单点或少数节点故障不影响整体）、去中心化。
缺点：消息延迟，存在一定的“谣言”传播时间，不能保证瞬间一致。

Redis Cluster 中的每个 Redis 实例监听两个 TCP 端口，6379（默认）用于服务客户端查询，16379（默认服务端口+10000）用于集群内部通信。集群中节点通信方式如下：

- 每个节点在固定周期内通过特定规则选择几个节点发送 Ping 消息；
- 接收到 Ping 消息的节点用 Pong 消息作为响应。

集群中每个节点通过一定规则挑选要通信的节点，每个节点可能知道全部节点，也可能仅知道部分节点，只要这些节点彼此可以正常通信，最终它们会达到一致的状态。当节点故障、新节点加入、主从关系变化、槽信息变更等事件发生时，通过不断的 Ping/Pong 消息通信，经过一段时间后所有的节点都会知道集群全部节点的最新状态，从而达到集群状态同步的目的。

小红书
小红书号：49607377662

分片数据如何传输

Redis Cluster分片数据的传输，这其实指的是在集群扩容或缩容时，**数据迁移（Data Migration）** 的过程。

它的核心是**以哈希槽为单位，通过原子性的MIGRATE命令进行传输，并辅以ASK重定向机制来保证客户端访问不受影响。”**

面试回答范例

“在 Redis Cluster 中进行数据迁移，比如将一个哈希槽从节点A迁移到节点B，整个过程被设计为**在线、原子且对业务影响最小化**的。它的核心流程可以分为三步：

1. **首先是准备与原子迁移阶段**：集群会通过命令分别将节点A和节点B上的这个槽标记为 `MIGRATING`（迁移中）和 `IMPORTING`（导入中）状态。随后，通过 `MIGRATE` 命令进行数据迁移。这个命令是**原子性的**，它会将键值对在节点A序列化、传输到节点B、节点B成功反序列化并返回确认后，节点A才会删除本地数据。这个 **‘确认后再删除’** 的机制，从根本上保证了在迁移过程中，任何一个键都不会丢失。
2. **其次是无缝的客户端重定向阶段**：这是保证服务可用的关键。在迁移过程中，客户端请求可能会遇到四种情况：
   - 键还在节点A，正常处理。
   - 键已被迁到节点B，节点A会返回 `-ASK` **重定向错误**，引导客户端去节点B临时访问。
   - 这里的关键在于，`-ASK` 是一个**一次性指令**，它告诉客户端‘下次去节点B试试’，但**不要求客户端更新本地的槽位映射缓存**。这与永久的 `-MOVED` 指令有本质区别。
   - 客户端在收到 `-ASK` 后，会先向节点B发送一个 `ASKING` 命令，然后执行原命令。这样就实现了业务访问与数据迁移的并发进行。
3. **最后是元数据更新与收尾阶段**：当槽内所有键都迁移完毕后，集群会通过 `CLUSTER SETSLOT` 命令，在集群所有节点上更新这个槽的归属信息，正式将其指派给节点B。此后，所有关于这个槽的请求，节点都会直接回复 `-MOVED` **指令**，客户端会根据这个永久重定向更新本地缓存，后续请求将直接发往节点B。

脑裂

在网络分区（脑裂）发生时，如何防止多个节点都认为自己是主节点，从而导致数据不一致。

> **问题描述**：旧主节点和它的从节点们因为网络问题被隔离成一个小集群。哨兵（Sentinel）或集群模式会在这个小集群之外选举出一个新的主节点。此时，系统中有两个主节点同时接受写请求，网络恢复后无法协调数据，导致数据混乱。

Redis 解决脑裂导致数据丢失

为了避免原主节点因网络故障单独Redis 提供了两个关键配置参数，作为解决脑裂和数据丢失的**最重要防线**：

- **连接的从节点数量** `min-replicas-to-write <N>`
  - 作用：主节点必须至少有 N 个从节点连接，才允许接受写请求。

小红书
小红书号：49607377662

    - 目的：避免数据写入后无法同步到其他节点。
- **复制延迟** `min-replicas-max-lag <T>`
    - 作用：如果主节点发现所有的从节点的复制延迟超过 T 秒，则禁止写入。
    - 目的：确保从节点数据是最新的，防止数据丢失。

扩缩容期间的可用性

Q: Redis Cluster 扩缩容期间是否可以持续提供服务？底层机制是什么？

A： 是的，可以持续提供服务。这是 Redis Cluster 的一个重要特性，其底层机制是 **原子性的槽迁移** 和 **ASK 重定向。**

底层机制：

1. **迁移单个 Key**：当将一个槽从节点 A 迁移到节点 B 时，迁移是以 **Key 为最小单位** 进行的。
2. **原子性操作**：对于正在迁移的单个 Key：
    - 在节点 A 上，这个 Key 会被设置为**迁移中**状态。
    - 然后，节点 A 会序列化这个 Key 并发送给节点 B。
    - 节点 B 成功接收并反序列化后，会给节点 A 发送一个确认。
    - 节点 A 收到确认后，才会在自己的数据库中**删除**这个 Key。
3. **ASK 重定向**：
    - 在迁移**过程中**，如果客户端访问节点 A 请求一个**已经被迁走**的 Key，节点 A 无法处理，但它知道这个 Key 现在在节点 B。
    - 节点 A 不会返回 MOVED 错误（因为迁移未完成，槽的归属权还未正式变更），而是返回一个 **ASKING** 错误，引导客户端去节点 B 访问。
    - 客户端收到 ASK 错误后，会向节点 B 发送一个 `ASKING` 命令，然后再次执行原命令。这次访问是**临时的**，客户端不会更新本地的槽映射缓存。

事务

Redis 事务

Redis支持事务，但其事务机制与传统关系型数据库的事务不同。Redis 的事务更像是 「打包执行」 而不是 「保证ACID」。
它主要目的是解决多条命令的网络往返开销和执行时的并发干扰，不保证严格的原子性和隔离性。

Redis 事务通过 `MULTI`、`EXEC`、`DISCARD`、`WATCH` 四个核心命令实现。

- 通过 `MULTI`、`EXEC`、`DISCARD` 进行事务管理，但 **不支持回滚**，如果事务中某个命令失败，其他命令仍会继续执行。

小红书
小红书号：49607377662

- `WATCH key` 机制：**通过** `WATCH` 可以实现 **CAS（Check And Set）** 乐观锁，防止并发写入冲突。

Redis 事务的适用场景

- 批量执行多个命令（如更新多个键）。
- 简单并发控制（结合 `WATCH` 实现乐观锁）。

事务执行流程

1. **开启事务**：使用 `MULTI` 命令开启事务，后续命令加入队列。
2. **命令入队**：将 `SET`、`INCR` 等命令缓存在队列中。
3. **执行事务**：`EXEC` 触发执行，按顺序执行所有命令。
4. **结果返回**：返回所有命令的执行结果数组（若事务被 `WATCH` 打断，返回 `nil`）。

和 MySQL 事务的区别

- **不支持回滚**，执行失败不会回滚之前的命令。
- **不支持部分成功**，如果 `EXEC` 时报错，已执行的指令不会撤回。

分布式锁 ✅

实现注意问题

使用Redis实现分布式锁时，必须考虑多个关键问题，

1. **加锁操作的原子性**：加锁必须是**原子操作**，否则多个客户端可能同时认为自己获得了锁；
2. **锁的持有者一致性**：释放锁时必须确保只有锁的持有者才能释放，避免误删其他客户端的锁；
3. **防止死锁**:锁必须设置过期时间，防止客户端崩溃后锁无法释放；
4. 在Redis集群环境下，主从复制存在延迟，可能导致多个客户端同时持有锁，因此需要Redlock等更复杂的算法来增强安全性；
5. 考虑实际应用中的重入、锁续期等高级特性

实现方式

小红书
小红书号：49607377662

1.SETNX 命令

使用 **SETNX命令**确保**只有一个线程能获取锁**

SETNX底层：Redis **底层存储**使用 **哈希表（dict）**，`SETNX` 操作会检查 key 是否存在，并尝试插入新值。

1. **获取锁**：使用 **SETNX** 尝试获取锁。
   - 使用SET命令加锁，设置NX（不存在才设置）和PX（过期时间）
     - 如果 key **不存在** → 插入并返回 `1`
     - 如果 key **已存在** → 直接返回 `0`，不覆盖
2. **设置过期时间**：防止因为程序崩溃或者锁没有被释放，导致死锁的发生

   例如，设置 **EXPIRE** lock_key 30使得锁在 30 秒后自动释放。
3. **释放锁**：释放锁时需要确认锁的值是否匹配。如果匹配，则删除锁

🟢优化： **SETNX+lua脚本**

- **原理**：通过 Lua 脚本将**多个 Redis 命令合并成一个原子操作**，确保锁的获取和释放的原子性
- **实现步骤**
  1. 执行 `SETNX` 获取锁。如果锁已存在，返回失败。
  2. 锁释放时，使用 Lua 脚本判断锁的拥有者，并删除锁。

这种方式可以确保**锁的操作是原子性的，避免多线程环境下的竞态条件。**

- **加锁**：`SET key value NX PX expire_time`
- **解锁**：使用 **Lua 脚本** 确保 `DEL` 之前先检查锁的 `value`

2.Redisson

Redisson 是一个基于 Redis 的 Java 客户端，它在**原生 Redis 分布式锁的基础上，做了更完善的封装。**

1. **原理**
   - 基于 Redis 的 **SETNX + EXPIRE** 实现互斥。
   - 使用 **Lua 脚本** 保证操作的 **原子性**（加锁、释放锁等动作不会被中断）。
   - 内置 **Watchdog 看门狗机制**：如果业务执行时间超过锁的过期时间，会自动续期，避免锁过早释放。
2. **解决了原生 Redis 分布式锁的三大痛点**
   - **自动续期**：原生锁过期时间写死，业务执行慢就会“误释放”；Redisson **通过 Watchdog 定时续约**，直到任务执行完成。
   - **可重入**：同一线程多次加锁时，Redisson 会用线程标识 + 重入计数，避免阻塞。
   - **避免死锁**：如果持有锁的实例宕机，Redisson 依靠锁过期 + Watchdog 自动释放，其他线程仍可继续获取。
3. **优点**
   - 开箱即用，开发者只需调用 `lock.lock()`、`lock.unlock()` 即可，不用关心底层逻辑。
   - 支持多种模式：公平锁、公平信号量、读写锁、红锁（Redlock，保证更强一致性）。
   - 更适合在 **微服务高并发场景** 使用。

✅ **关键点**

小红书
小红书号：49607377662

- **核心命令**
  - `SETNX` **(SET if Not Exists)**
    - 如果 key **不存在**，则设置锁并返回成功 → 保证 **原子性加锁。**
    - 如果 key 已存在 → 加锁失败，线程可选择重试或等待。

    `EXPIRE`
    - 给锁设置 **过期时间**，避免业务执行异常或线程挂掉导致死锁。
    - Redisson 还通过 **watchdog** 自动续期，确保长业务不会被误释放
- **WatchDog 机制**：如果加锁时**未指定超时时间**，Redisson会启动一个看门狗线程。
  - **问题**：如果业务执行时间超过了锁的默认超时时间 (`leaseTime`)，锁会自动释放，导致临界区被多个线程进入，失去安全性。
  - **工作流程**：看门狗默认每隔 **10 秒** (`lockWatchdogTimeout` / 3) 检查一下客户端是否还持有锁。如果还持有，就通过 Lua 脚本**刷新锁的超时时间**（续期），重置为默认的 30 秒。
  - **意义**：只要 JVM 进程没有宕机，且业务没有执行完，看门狗就会一直给锁“续命”，从根本上避免了因业务执行时间长而导致的锁超时问题。
- **Lua 脚本**保证原子操作，确保**加锁、续期、释放锁的安全性。**
  - **加锁**：执行Lua脚本，用 `HASH` 结构存储锁信息（锁名称、持有线程ID、重入次数），并设置过期时间。
  - **释放锁**：同样通过Lua脚本实现，先判断锁属于当前线程（value 匹配），然后减少重入次数，次数为0则删除KEY。
- **可重入支持**：每个锁会维护一个 **计数器（reentrant count）**：
  - 同一个线程重复加锁时，计数器 +1。
  - 解锁时计数器 >1，则只减计数，不删除 key。
  - 计数器为0时才删除 Redis key。

RedLock

Redlock算法用于多实例Redis的分布式锁，用于**高可用场景**，Redis集群环境下通过多节点部署提高容错性（需至少5个主节点）。

核心思想：

1. 在 **N 台 Redis 实例**（一般 5 个）上尝试加锁。
2. 如果 **至少有** `N/2+1` **个实例成功加锁**，则认为加锁成功。
3. 计算锁的持有时间，若超过超时时间，锁失效

限流

Redis 限流核心思想

利用 Redis 的两个关键特性：

1. **单线程执行**：所有命令串行执行，天然避免了并发问题，可以轻松实现原子性的计数操作。
2. **高性能与过期时间**：读写速度极快，并且可以方便地设置 key 的过期时间，从而实现时间窗口。

利用 Redis 做分布式限流，主要是依赖其单线程原子操作的特性。

小红书
小红书号：49607377662

限流算法

固定窗口计数器

这是最简单、最直接的限流算法。

- **原理**：将一个时间段作为一个固定的“时间窗口”，在这个窗口内对请求计数，如果计数超过阈值，则拒绝后续请求。下一个时间窗口开始时，计数器清零。
- **Redis 实现**：
  - **Key**：`rate_limit:api:user_1:window_1717000000`（由限流目标 + 窗口起始时间戳构成）
  - **命令**：`INCR` + `EXPIRE`
  - **Lua 脚本（保证原子性）**：

漏桶算法

原理：水（请求）倒入漏桶中，桶以固定速率出水。

特点：流量恒定、平滑，适合**削峰**。

实现：

- 在 Redis 里用计时器（时间戳）+ 剩余容量模拟。
- 每次请求时计算漏掉多少水，然后更新桶容量。
- 如果桶满了（请求数超过容量），拒绝请求。

令牌桶算法(最通用)

这是生产环境中最常用、最平滑的限流算法。**Redis 4.0 以上版本提供了 `redis-cell` 模块**，直接实现了令牌桶算法。

- **原理**：
  1. 以一个固定的速率（`r tokens/s`）向一个容量为 `b` 的桶里添加令牌。
  2. 当请求到来时，需要从桶中获取一个令牌。如果拿到令牌，则处理请求。
  3. 如果桶中没有令牌，则请求被限流
- **为什么这么设计**：
  - **允许突发流量**：只要桶里有令牌，突发请求可以一次性处理，这对于短时高并发场景很友好。
  - **平滑流量**：长期来看，请求的处理速率被稳定在`r`，保护了下游系统。
  - 这是生产环境中最常用、最灵活的限流算法之一。

它的核心实现思路是通过一个 Lua 脚本原子性地完成以下步骤：

1. 根据当前时间和上次更新时间，计算出应该补充多少令牌；
2. 更新当前桶内的令牌数（不能超过桶容量）；
3. 判断当前请求是否能获取令牌，并返回结果。

使用

小红书
小红书号：49607377662

在生产中，我们一般不会自己手写这个脚本，而是采用两种更成熟的方式：
一是使用 **Redisson 的** `RRateLimiter`，开箱即用，API 简单，支持集群模式，性能优异。
二是为 Redis 安装 `redis-cell` **模块**，它直接提供了一个 `CL.THROTTLE` 命令，这是最彻底的解决方案，所有语言的客户端都能直接调用，性能也是最好的。

基于 Redis 的分布式限流

Redis 的INCR（原子自增）和EXPIRE（过期时间）特性，天然适合分布式计数，核心逻辑：

1. 用 Redis 键（比如limit:api:user）表示某接口的限流标识；
2. 每接收 1 个请求，执行INCR key，并判断自增后的值是否超过阈值；
3. 若未超过，正常处理；若超过，拒绝请求；
4. 为键设置过期时间（比如 1 秒），避免键堆积。

为避免 “先 INCR 再 EXPIRE” 的原子性问题（比如 INCR 后服务宕机，键未设置过期），需用**Lua 脚本**保证操作原子性，代码示例（Redis Lua）

共性问题

无论采用哪种方案，基于Redis的限流器都会面临一些共性问题：

1. **原子性问题 (Atomicity)**
   - **问题**：在高并发下，`INCR`和`EXPIRE`分开执行，或判断和操作分开执行，会导致竞态条件。
   - **解决**：**必须使用Lua脚本**将关键逻辑封装成一个原子操作执行。
2. **时钟漂移问题 (Clock Drift)**
   - **问题**：在令牌桶等算法中，如果Redis服务器的时间和应用程序时间不同步，会导致计算错误。
   - **解决**：尽量使用Redis服务器的时间（在Lua脚本中用`redis.call('time')`获取），或者使用NTP协议保证所有节点时间同步。
3. **Redis单点压力 (Single Point of Pressure)**
   - **问题**：所有请求都要访问Redis进行限流判断，Redis本身可能成为瓶颈和单点故障。
   - **解决**：
     - **集群模式**：使用Redis Cluster将不同限流Key分片到不同节点上，分散压力。
     - **本地缓存+同步**：对于网关等层面的限流，可以先在本地做一层粗粒度限流，减少对Redis的请求。或者像**令牌桶**算法，可以一次预取一批令牌到本地，在本地消费，消费完再去Redis获取，大幅减少网络请求。
4. **限流键 (Key) 的设计**
   - 需要根据业务灵活设计。常见的维度有：`用户ID`、`IP地址`、`接口名`等。例如：`rate_limit:user:123:api_write`。

小红书
小红书号:49607377662

3.Zookeeper

“Zookeeper 是一个分布式协调服务，常用于配置管理、服务注册发现和分布式锁。”

ZooKeeper的典型应用场景包括**分布式锁**、配置管理、Leader选举等。

比如在分布式锁中，可以通过创建临时顺序节点来实现排他锁或共享锁。

当客户端想要获取锁时，创建一个临时顺序节点，并检查自己是否是序号最小的节点，如果是则获得锁，否则监听前一个节点的删除事件。这种方式能有效避免死锁，

Zookeeper 使用 **临时有序节点** 实现锁：

1. **客户端创建临时有序节点** `/lock/lock_00001`，并监视比自己序号小的节点。
2. **最小序号的节点获得锁**，完成任务后删除节点。
3. **下一个最小节点获得锁**，形成**队列模式**，避免 Redis 锁竞争的问题。

分布式锁如何保证可靠性?

分布式锁核心特性

一个可靠的分布式锁需具备以下特性：

1. **互斥性**：同一时刻仅有一个客户端持有锁。
2. **防死锁**：锁需设置超时时间（TTL），避免持有者宕机导致锁永久不可用。
3. **容错性**：即使部分节点故障，锁服务仍能正常运作。
4. **可重入性**：同一线程可多次获取同一锁。
5. **高性能与高可用**：获取/释放锁的操作需高效，锁服务需具备高可用性（如Redis Cluster）

分布式锁的可靠性非常重要。它需要保证在多个节点之间的协调、锁的持有、超时、死锁等问题时刻得以处理。

1.**锁超时机制**

- **加锁超时：** 为了防止死锁的发生，锁会被赋予一个超时时间。如果超时没有释放锁，锁会被自动释放
- **续期机制：** 在某些场景下，持有锁的任务可能需要较长时间才能完成
- > 对锁设置有效期（例如 30 秒），并在任务执行过程中定时检查是否需要延长锁的持有时间。
  >
  > 在分布式锁中，通过 `SETNX` 和 `EXPIRE` 命令设置锁和超时时间。
  >
  > 比如 Redis 中的 `Redisson` 就使用了 Watchdog 机制来自动续期锁

小红书
小红书号：49607377662

2.锁的互斥性

互斥性是指同一时刻只有一个客户端能够获得锁。

- > Redis 中，`SETNX` 命令能够确保一次只有一个客户端能成功获取到锁。

3.防止锁重入

- 防止锁重入是为了避免锁被同一个客户端重复持有，导致资源的不公平竞争。

> Redis 的 `SETNX` 命令或 Redisson 提供的锁重入功能。
>
> Redisson 支持锁的重入，但是它会检查当前线程是否已经持有该锁，如果是，则不会发生阻塞。

4.锁的可重入性（对于同一线程）

- 锁的可重入性:为了避免一个线程在持有锁的过程中再次尝试获取同一锁，造成死锁。
- Redisson 就提供了对锁重入的支持，当同一个线程再次请求锁时，Redisson 会允许其重复获取，并且不会造成死锁。

优化

优化查询和内存

优化 Redis 查询速度

1. **使用合适的数据结构**（如 HyperLogLog、BitMap、ZSet）。
2. **减少 Key 数量**，使用 Hash 结构存储类似数据。
3. **开启 Lazy Free 机制**，异步释放内存，减少阻塞。
4. **使用 Pipeline 进行批量操作**，减少 RTT（往返延迟）。
5. **使用 Lua 脚本**，在 Redis 端执行逻辑，避免多次网络请求。

优化 Redis 内存使用

1. **合理设置过期时间**，清理无用数据（`EXPIRE`）。
2. **启用 LRU / LFU 过期策略**，淘汰冷数据（`maxmemory-policy`）。

小红书
小红书号:49607377662

缓存或数据库宕机

缓存宕机：

- 如果只是部分节点宕机，可以通过一致性哈希或代理层让请求路由到其他节点；
- 如果整个缓存集群宕机，请求会直接打到数据库，压力骤增。这时需要 **限流和降级**，比如用令牌桶算法限制请求量，对非核心请求返回静态数据或默认值，避免数据库过载。同时可以通过后台任务 **异步重建缓存**，逐步加载热点数据恢复服务。

数据库宕机：

- 如果是部分节点不可用，可以通过主从切换或数据库中间件做 **快速故障转移**；
- 如果整个数据库都挂了，可以考虑切换为 **只读模式**，依赖缓存中的数据提供有限服务。
- 平时则要通过 **持久化、备份和集群部署** 来提升数据库的可用性。

缓存宕机场景

1. 直接穿透到数据库

- **解决方案**：
  - **限流降级**：**接口层限流（如令牌桶算法）**，仅允许部分请求访问数据库。
  - **服务降级**：返回缓存中预存的兜底数据（如商品默认描述）。

2. 缓存集群部分节点宕机

- **解决方案**：**一致性哈希**将请求转移到其他健康节点

> 一致性哈希常用于分布式系统，比如缓存或数据库分片，它通过哈希环和虚拟节点来分配数据，减少节点变动时的数据迁移。
>
> 传统的哈希算法在节点数量变化时，几乎所有数据都需要重新哈希，导致大量数据迁移，影响系统性能。

Redis主库故障的写操作

- **写操作有效性**：若使用异步复制，主库宕机可能导致未同步的写操作丢失。
- **解决方案：**
  1. **WAIT命令**：同步写操作到至少N个从库（强一致性，性能低）。
  2. **Redis Sentinel/Cluster**：自动故障转移，客户端重试写入新主库。
  3. **客户端缓存写入日志**：故障恢复后重放日志。

小红书
小红书号：49607377662

数据库宕机场景

主库宕机会影响写操作，从库宕机影响读操作

1. 主库宕机（写操作不可用）：主从切换：自动切换从库为主库。

2.从库宕机（读服务不可用）

1.
    - 读流量转移
        - 将读请求分发至其他从库或临时指向主库（需评估主库负载）。
    - 缓存兜底：
        - 优先返回 **Redis缓存** 的热点数据（如最近1小时商品详情）。
        - 缓存穿透时返回静态默认值（如“库存充足”），而非直接报错。

热点Key问题 📌

热点Key问题，也称为**热Key问题**，是指系统中某个特定的Redis Key接收到的访问量（QPS）远高于其他Key，导致该Key所在的Redis实例负载过高，成为系统性能瓶颈。

- 成因：
    - 业务场景集中：如双11商品详情页、微博热搜榜Top1。
    - 缓存设计不合理：将高频访问的数据集中存储在一个key中（如用一个hash存所有用户配置）。
    - 缓存穿透/雪崩后集中重建缓存，导致某个key被频繁查询数据库并回填。
- 影响：
    - 单点过载：Redis是单线程处理命令（网络IO多线程从Redis 6.0开始），一个热key的高并发访问会阻塞其他请求处理。
    - 网络带宽耗尽：大量请求和响应占用网络资源。
    - 主从复制延迟：主节点压力大，同步到从节点的数据延迟增加。
    - 集群中节点不均衡：在Redis Cluster中，热key集中在某个slot，导致该slot所在节点成为瓶颈

解决方案

小红书
小红书号：49607377662

写热点：MQ异步

对于像“秒杀库存”这样的**写热点**，

- **原子操作+Lua脚本**：扣减库存时，必须使用`INCR`/`DECR`或Lua脚本，保证原子性，避免超卖。
- **串行化**：将并发的写请求放入消息队列（如Kafka、RocketMQ）中，由Worker顺序消费，将其转化为串行操作，极大减轻Redis压力。

读热点：本地缓存

这是解决**读热点**最立竿见影的方案。

- **方案**：使用Guava Cache、Caffeine或Ehcache等本地缓存组件，将热Key的数据缓存在**应用进程的内存**中。
- **细节**：
  - 设置一个**短的过期时间**（如1-3秒），防止本地数据与Redis数据长期不一致。
  - 当本地缓存失效后，只有一个请求会去Redis加载数据，其他请求继续使用旧数据，从而避免大量请求同时击穿Redis。
- **效果**：99%的读请求根本不会到达Redis，直接在本地就被处理了。50万QPS的热点请求，落到Redis上的可能只有几百QPS。
- **代价**：存在数据不一致的风险（短时间），需要根据业务场景权衡。

Key分片（Hashing）

- **方案**：将热Key复制多份，变成多个子Key。
  - 例如，热Key原是`hotkey:1001`，可以复制为`hotkey:1001:1`, `hotkey:1001:2`, ... `hotkey:1001:10`。
  - 读取时随机选择一个分片读取（例如，对用户ID取模）。
  - 写入时需更新所有分片（一致性要求高时）或只写Redis+失效本地缓存。
- **效果**：将对一个Key的访问压力，分散到多个Key上，从而分散到Redis集群的不同节点上。
- **缺点**：更新数据时需要同时更新所有子Key，操作变得复杂。

架构优化

- **使用Redis集群**：确保热Key通过哈希槽分散到不同的物理实例上（尽管可能无法完全解决，但能避免一个实例宕机导致全盘皆输）。
- **提升硬件**：如果热Key无法避免，临时提升该实例的机器规格（如带宽升级到10G）、使用读写分离从节点分担读压力，作为应急手段。

小红书
小红书号：49607377662

Redis 宕机如何恢复数据

首先看是否开启了 AOF，Redis 优先加载 AOF 文件，否则加载 RDB 快照恢复。如果有备份，可以拷贝备份文件恢复。

在主从或 Sentinel 架构下，可以直接切换从节点为主节点，保证业务不中断。

恢复后确认数据完整性，分析是否有数据丢失，结合业务日志做补偿。同时要分析宕机根因，优化配置，避免再次出现。

平时通过 **AOF+RDB 双持久化**、哨兵或集群高可用、监控和限流降级方案保证 Redis 的稳定性和数据安全。

Q&A

✅ **Redis 常见数据结构?**

- `String`（缓存、计数器）、`Hash`（存储对象）、`List`（消息队列）、`Set`（去重）、`ZSet`（排行榜）。

✅ **Redis 的持久化方式?**

- **RDB（快照）**：周期性生成 `.rdb` 文件，适合冷备份。
- **AOF（日志）**：记录所有写操作，恢复时逐条执行，数据更完整。

✅ **Redis 分布式锁如何实现?**

- **基于 SETNX + EXPIRE**：`SET key value NX PX 30000`，加锁并设置超时时间。
- **Redisson 实现分布式锁**：基于 Lua 脚本和 WatchDog 机制，自动续期。
- **Zookeeper 实现分布式锁**：基于临时有序节点。

✅ **缓存击穿、缓存穿透、缓存雪崩怎么解决?**

- **缓存穿透**（查询不存在的数据）：使用 **布隆过滤器**，拦截非法请求。
- **缓存击穿**（热点 Key 失效）：**加互斥锁（Redis 分布式锁）**，防止大量请求直接打到数据库。
- **缓存雪崩**（大量 Key 同时失效）：设置 **不同的过期时间**，避免同时失效。

---

3. Redis 分布式架构

✅ **Redis 如何实现主从复制?**

- **主从同步（Replication）** 通过 `SLAVEOF` 命令或 `replicaof` 配置项实现。
- **全量复制**（初次同步）：主节点生成 RDB 发送给从节点。
- **增量复制**（断点续传）：主节点发送 `repl_backlog_buffer` 中的数据，减少同步成本。

✅ **Redis 如何实现高可用？（哨兵模式 Sentinel）**

小红书
小红书号：49607377662

- **监控（Monitoring）**：Sentinel 监控主节点，检测是否故障。
- **自动故障转移（Failover）**：如果主节点挂了，Sentinel 会选举一个新的主节点。
- **通知（Notification）**：Sentinel 通过发布订阅通知客户端新的主节点信息。

✅ **Redis Cluster 分片原理？**

- **数据分片**：基于 **哈希槽（Hash Slot）**，将 16384 个哈希槽均匀分配到不同节点。
- **主从架构**：每个分片有主节点（负责读写）和从节点（负责备份）。
- **请求路由**：客户端连接任意一个节点，节点通过 **MOVED** 命令引导请求到正确的分片。

✅ **Redis Cluster 如何实现高可用？**

- **数据冗余**：主从结构保证数据不丢失。
- **节点自动迁移**：主节点故障后，从节点可自动提升为主节点（选举方式类似 Sentinel）。
- **Gossip 协议**：节点间通过 Gossip 传播状态信息，检测故障并触发主从切换。

4. Redis 事务与分布式锁

✅ **Redis 事务如何实现？支持回滚吗？**

- 通过 `MULTI`、`EXEC`、`DISCARD` 进行事务管理，但 **不支持回滚**，如果事务中某个命令失败，其他命令仍会继续执行。
- `WATCH key` 机制：可以实现 **CAS（Check And Set）** 乐观锁，防止并发写入冲突。

✅ **Redis 如何实现分布式锁？**

- 方案 1：基于 SETNX + EXPIRE

  （单机锁，存在死锁风险）
- 方案 2：Redisson（推荐，支持自动续期、可重入锁）
  - 使用 Lua 脚本保证 **加锁 + 过期时间原子性**，防止锁误删。
  - WatchDog 机制自动续期，防止业务超时导致锁提前释放。

1. Redis 如何实现原子性？

- **单线程模型**：所有命令顺序执行，天然原子性。
- **Lua 脚本**：将多个操作封装为脚本，原子执行（推荐）。
- **事务（MULTI/EXEC）**：命令队列顺序执行，但无回滚机制（部分失败需补偿）。

小红书
小红书号：49607377662

2. Redis是单线程的吗？

- **网络I/O和数据读写**：是**单线程**的。这意味着所有命令的接收、执行、返回都是顺序串行执行的，避免了不必要的上下文切换和竞态条件，使得实现简单、性能极高。
- **其他功能**：是**多线程**的。
  - **Redis 4.0**：引入多线程用于在后台执行**惰性删除**（`UNLINK`，`FLUSHALL ASYNC`）等大Key删除操作，不阻塞主线程。
  - **Redis 6.0+**：引入**I/O多线程**。**注意**：这里的多线程只用于处理网络，**命令的执行依然由主线程串行进行**。这对于应对超高带宽的网络场景非常有效。

为什么命令执行还是单线程？
因为Redis的性能瓶颈通常不在CPU，而在于**内存和网络**。单线程模型避免了锁的开销，保证了原子性，已经足够高效。

3. 如何保证 Redis 数据一致性？内存不足怎么办？

数据一致性策略：
- **双删策略**：先更新数据库 → 删除缓存 → 延迟再次删除缓存。
- **消息队列补偿**：异步监听数据库变更，更新缓存。

内存不足处理：
- **淘汰策略**：配置 `maxmemory-policy`（如 `volatile-lfu`）。

```
LRU：淘汰最近最少使用的数据。
LFU：淘汰访问频率最低的数据（适合低频访问场景）。
TTL：优先淘汰即将过期的键。
```

- **惰性删除**：访问时检查过期时间（默认）。
- **主动删除**：定期随机清理过期键。

4. Redis 为什么效率高？详细解释 I/O 多路复用

回答： Redis 的高效性主要基于以下设计：

1. **单线程模型**：避免多线程上下文切换和锁竞争，所有命令按顺序原子执行。
2. **内存存储**：数据直接读写内存，避免磁盘 I/O 瓶颈。
3. **I/O 多路复用**：
   - **核心机制**：通过 `epoll/select/kqueue` 监听多个客户端连接的事件（读/写就绪）。
   - **非阻塞 I/O**：单线程轮询处理就绪的请求，避免线程阻塞。
   - **优势**：单线程处理 10 万+并发连接，资源占用低，吞吐量高。

小红书
小红书号：49607377662

5. Redis 底层数据结构 & Sorted Set 实现

回答：

1. **底层数据结构**：
   - **String**：SDS（动态字符串，预分配内存减少频繁扩容）。
   - **Hash**：压缩列表（小数据）或哈希表（大数据）。
   - **Sorted Set**：
     - **跳跃表（SkipList）**：多层链表加速范围查询（平均 O(logN)）。
     - **字典（Hash Table）**：存储成员到分数的映射（O(1) 查找）。

2. **跳跃表新增索引步骤**：
   - **随机层高**：通过抛硬币算法随机生成新节点的层数（如 1~32 层）。
   - **逐层插入**：从最高层开始，找到每层插入位置，更新前后指针。
   - **示例**：插入节点时，若随机层高为 3，则在第 1~3 层链表中插入该节点。

6. 分布式锁实现 & 超卖场景

回答：

1. **分布式锁实现（Redisson）**：
   - **加锁**：使用 Lua 脚本原子执行 `SETNX key value PX timeout`。
   - **看门狗机制**：后台线程自动续期锁（默认 30 秒过期，每 10 秒续期）。
   - **释放锁**：校验锁的 value（客户端唯一标识），防止误删其他客户端的锁。
2. **超卖场景解决方案**：
   - **数据库行锁**：`SELECT ... FOR UPDATE` 锁定库存行（悲观锁）。
   - **Redis 原子操作**：通过 `DECRBY` 或 Lua 脚本保证原子扣减。

   对比：大厂直接数据库锁简化架构，但 Redis 方案性能更高（需结合降级策略）。

小红书
小红书号：49607377662